ADR 0008 - Eval judge protocol

Status: Deferred **Date**: 2026-06-23 **Author**: Ronin **Related**: docs/SCORING.md, docs/DEFERRED.md, ADR 0003

Context

BAYMAX evals need to decide whether an agent response satisfied a scenario. Some behaviors are easy to score deterministically:

```
expected tool == actual tool
expected start_time == actual start_time
expected body_contains appears in actual body
no tool call was made for clarification/refusal
```

Other behaviors are semantically messier. A clarification question can be worded many ways. A refusal can be polite, brief, or detailed while still being correct. Later multi-turn scenarios may require judging whether context was preserved across turns.

An LLM-as-judge protocol could score these softer cases, but it would introduce another model dependency into Phase 1 and make results less reproducible.

Decision

Phase 1 does **not** use LLM-as-judge. The scorer remains deterministic.

Current deterministic checks:

```
tool-call accuracy          ordered expected tools vs actual tools
unordered tool match        right tools in wrong order visibility
argument accuracy           exact arguments and explicit *_contains checks
clarification accuracy      required keywords in response message, no tool calls
refusal accuracy            required keywords in response message, no tool calls
hallucination rate          actual tools not listed in available_tools
task success                no failure reasons and full argument correctness
```

LLM-as-judge is deferred until deterministic checks become too brittle for the scenario set. When added, it should be a separate scorer mode, not a replacement for deterministic scoring.

Reasoning

why chosen - v1 needs benchmark numbers that are cheap, reproducible, and easy to debug. A judge model would create a second source of nondeterminism: the agent might fail, or the judge might disagree with the rubric. Deterministic scoring is less flexible but much easier to trust while we are still validating the scenario format, fake adapters, runner, and baseline output. Keeping judge work deferred also prevents Phase 1 from expanding into prompt-rubric engineering before the core harness is complete.

Alternatives considered

Alternative A - LLM judge from Phase 1

Use GPT/Claude as a judge for all natural-language responses and maybe tool-call reasoning.

why rejected - adds cost, latency, nondeterminism, prompt maintenance, and judge-version drift. It also makes local repeatability weaker before the harness is stable.

Alternative B - Human review only

Humans inspect scenario outputs and decide whether each run passes.

why rejected - useful for PR review but not a benchmark. It cannot produce repeatable aggregate metrics or CI-friendly failure reports.

Alternative C - Exact string matching only

Require the response message and arguments to exactly match expected strings.

why rejected - too brittle. Email bodies, titles, and clarification messages need limited flexibility. The `_contains` convention gives controlled flexibility without semantic judging.

Alternative D - Replace deterministic scorer with judge later

Use deterministic scoring in v1, then remove it once judge scoring exists.

why rejected - deterministic scoring remains valuable as a stable regression suite. Future judge scoring should be additive and used only where deterministic checks are insufficient.

Consequences

Positive

- Phase 1 results are reproducible and cheap
- Failures are easy to inspect because failure reasons are explicit
- CI can run the scorer without model access
- Scorer behavior is transparent enough to debug scenario authoring mistakes

Negative

- Semantically correct but differently worded responses can fail keyword checks
- `_contains` checks are weaker than true semantic equivalence
- Multi-turn and nuanced safety cases will eventually need richer judging

Neutral

- Scenario authors must choose clarification/refusal keywords carefully
- Future judge mode needs its own rubric, calibration set, and cost reporting

---

Open questions / follow-ups

- What scenario types justify judge scoring first: ambiguous clarification, refusal, or multi-turn?
- Which model judges the judge protocol, and how do we version judge prompts?
- Should judge scores be reported separately from deterministic scores?
- What minimum agreement with human review is required before judge scores are trusted?

References

- docs/SCORING.md
- docs/DEFERRED.md
- ADR 0003 - Scenario format and eval fixtures
- `src/baymax/eval/scorer.py`
- `results/phase1/scripted/v1-baseline.json`

Exported 2026-10-17. Tables and saved measurements are preserved. Combined PDFs use internal document navigation. Source/result links point to the GitHub branch; unpublished local data/checkpoint links are not active.